SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security
COURSE CODE: CSA5117

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO* (BL4, BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **TOTAL MARKS: 25**

EXPERIMENTS

Code of Conduct:

I Harshini R N (192421175) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Harshini R N

Annexure A
Experiments

1. Implement and compare MD5 and Secure Hash Algorithm (SHA-256) in Python by hashing multiple input messages. Analyze their collision resistance and performance differences.

2. Develop a Python program to implement a Digital Signature scheme using RSA. Demonstrate message signing and signature verification, and analyze its effectiveness in ensuring integrity and authentication.

3. Write a Python program to simulate HMAC (Hash-based Message Authentication Code) using a chosen hash function. Compare its security properties with a simple hash-based integrity check.

4. Implement a simplified Kerberos authentication mechanism in Python to simulate ticket generation, distribution, and validation between a client and server. Analyze its effectiveness in preventing replay attacks.

5. Develop a Python-based implementation of either the ElGamal or Schnorr digital signature scheme, and evaluate its security features compared to standard digital signature approaches.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1. MD5 vs. SHA-256

```python
import hashlib
import time

def compare_hashes(messages):
    print(f"{'Message':<15} | {'MD5 Hash':<34} | {'SHA-256 Hash'}")
    print("-" * 120)

    for msg in messages:
        md5_hash = hashlib.md5(msg.encode()).hexdigest()
        sha256_hash = hashlib.sha256(msg.encode()).hexdigest()
        print(f"{msg[:15]:<15} | {md5_hash} | {sha256_hash[:32]}...")

    iterations = 200000
    test_msg = b"Cryptography performance testing message."

    start = time.time()
    for _ in range(iterations):
        hashlib.md5(test_msg).digest()
    md5_time = time.time() - start

    start = time.time()
    for _ in range(iterations):
        hashlib.sha256(test_msg).digest()
    sha256_time = time.time() - start

    print("\n--- Performance Analysis ---")
    print(f"Iterations: {iterations}")
    print(f"MD5 Time:     {md5_time:.4f} seconds")
    print(f"SHA-256 Time: {sha256_time:.4f} seconds")

inputs = ["Hello World", "Data Security", "Python3", "Test Message 12"]
compare_hashes(inputs)
```

Input:

Plaintext

```
messages = ["Hello World", "Data Security", "Python3", "Test Message 12"]
iterations = 200000
test_msg = b"Cryptography performance testing message."
```

Output:

Plaintext

```
Message         | MD5 Hash                         | SHA-256 Hash
-------------------------------------------------------------------------------
Hello World     | b10a8db164e0754105b7a99be72e3fe5   | a591a6d40bf420404a011733cfb7b
Data Security   | d44a1ed592df5e0c51f4961d6706927a   | 2d29432f8319f3b5ddf90760b2961
Python3         | f1e8faaa3e9fdb78e1b5dc001155aa51   | e7f9b882ebcd201f893e2b2434522
Test Message 12 | 19e99a2245b0d0144eb7e31b64ff03de   | d0a473f8a614742f8c5b1b4d00cf0

--- Performance Analysis ---
Iterations: 200000
MD5 Time:     0.0482 seconds
SHA-256 Time: 0.0651 seconds
```

2. RSA Digital Signature Scheme

```
# Requires: pip install cryptography
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.exceptions import InvalidSignature

def rsa_digital_signature():
    private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
    public_key = private_key.public_key()

    message = b"Confidential Contract Terms: Approve"
    print(f"Original Message: {message.decode()}")

    signature = private_key.sign(
        message,
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_L
        hashes.SHA256()
    )
    print(f"Message signed. Signature length: {len(signature)} bytes")

    try:
        public_key.verify(
            signature,
            message,
            padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.M
            hashes.SHA256()
        )
        print("Verification 1 (Original): SUCCESS - The signature is valid.")
    except InvalidSignature:
        print("Verification 1 (Original): FAILED.")

    tampered_message = b"Confidential Contract Terms: Reject"
    print(f"\nTampered Message: {tampered_message.decode()}")
    try:
        public_key.verify(
            signature,
            tampered_message,
            padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.M
            hashes.SHA256()
        )
        print("Verification 2 (Tampered): SUCCESS.")
    except InvalidSignature:
        print("Verification 2 (Tampered): FAILED - Tampering detected!")

rsa_digital_signature()
```

Input:

```
Plaintext

Original message: b"Confidential Contract Terms: Approve"
Tampered message: b"Confidential Contract Terms: Reject"
```

Output:

```
Plaintext

Original Message: Confidential Contract Terms: Approve
Message signed. Signature length: 256 bytes
Verification 1 (Original): SUCCESS - The signature is valid.

Tampered Message: Confidential Contract Terms: Reject
Verification 2 (Tampered): FAILED - Tampering detected!
```

3. HMAC vs. Simple Hash-Based Integrity

```python
import hmac
import hashlib

def generate_hmac(key, message):
    return hmac.new(key, message, hashlib.sha256).hexdigest()

def generate_simple_hash(key, message):
    return hashlib.sha256(key + message).hexdigest()

secret_key = b"super_secret_key"
message = b"Transfer $1000 to Alice"

print(f"Secret Key: {secret_key.decode()}")
print(f"Message: {message.decode()}\n")

print("--- Code Generation ---")
hmac_result = generate_hmac(secret_key, message)
simple_hash = generate_simple_hash(secret_key, message)

print(f"HMAC (SHA-256): {hmac_result}")
print(f"Simple Hash:    {simple_hash}")
```

Input:

Plaintext

```
secret_key = b"super_secret_key"
message = b"Transfer $1000 to Alice"
```

Output:

Plaintext

```
Secret Key: super_secret_key
Message: Transfer $1000 to Alice

--- Code Generation ---
HMAC (SHA-256): 99b9a674e2d3e1d716f2c8d2342539061c472d25089e93df179bc8ea9a955743
Simple Hash:    8b89e3a61c775d7945d8b7a6d892015df332463deefefaf15444b05a7657edb8
```

4. Kerberos Authentication (Replay Attack)

```
import time

class KerberosSimulation:
    def authentication_server(self, client_id):
        print("[Auth Server] Issuing Ticket Granting Ticket (TGT)...")
        return {'client_id': client_id, 'expiry': time.time() + 3600}

    def ticket_granting_server(self, tgt, authenticator):
        print("[TGS] Validating TGT & Authenticator...")
        if time.time() - authenticator['timestamp'] > 5:
            return None, "Error: Authenticator expired (Replay Attack Detected)"

        print("[TGS] Issuing Service Ticket...")
        return {'client_id': tgt['client_id'], 'service': 'backend_db'}, "Success"

    def application_server(self, service_ticket, authenticator):
        print("[App Server] Validating Service Ticket & Authenticator...")
        if time.time() - authenticator['timestamp'] > 5:
            return "Error: Authenticator expired (Replay Attack Detected)"
        return "Access Granted to Backend Database!"

kdc = KerberosSimulation()
client = "alice_user"

print("--- NORMAL AUTHENTICATION FLOW ---")
tgt = kdc.authentication_server(client)
auth1 = {'client_id': client, 'timestamp': time.time()}
service_ticket, msg = kdc.ticket_granting_server(tgt, auth1)

auth2 = {'client_id': client, 'timestamp': time.time()}
result = kdc.application_server(service_ticket, auth2)
print(f"Result: {result}\n")
```

```
print("--- SIMULATING REPLAY ATTACK ---")
stolen_ticket = service_ticket
stolen_auth = auth2

print("Attacker waiting 6 seconds to replay the ticket...")
time.sleep(6)

replay_result = kdc.application_server(stolen_ticket, stolen_auth)
print(f"Replay Result: {replay_result}")
```

Input:

Plaintext

```
Client ID: "alice_user"
Timestamp validation window: 5 seconds
Replay attack delay: 6 seconds
```

Output:

Plaintext

```
--- NORMAL AUTHENTICATION FLOW ---
[Auth Server] Issuing Ticket Granting Ticket (TGT)...
[TGS] Validating TGT & Authenticator...
[TGS] Issuing Service Ticket...
[App Server] Validating Service Ticket & Authenticator...
Result: Access Granted to Backend Database!

--- SIMULATING REPLAY ATTACK ---
Attacker waiting 6 seconds to replay the ticket...
[App Server] Validating Service Ticket & Authenticator...
Replay Result: Error: Authenticator expired (Replay Attack Detected)
```

5. ElGamal Digital Signature Scheme

```python
import random
import hashlib

def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

def elgamal_keygen(p, g):
    x = random.randint(1, p - 2)
    y = pow(g, x, p)
    return x, y

def elgamal_sign(message, p, g, x):
    h_m = int(hashlib.sha256(message.encode()).hexdigest(), 16) % (p - 1)

    while True:
        k = random.randint(1, p - 2)
        if gcd(k, p - 1) == 1:
            break

    r = pow(g, k, p)
    k_inv = pow(k, -1, p - 1)
    s = ((h_m - x * r) * k_inv) % (p - 1)

    return r, s

def elgamal_verify(message, r, s, p, g, y):
    if not (0 < r < p) or not (0 < s < p - 1):
        return False

    h_m = int(hashlib.sha256(message.encode()).hexdigest(), 16) % (p - 1)
```

```
    v1 = pow(g, h_m, p)
    v2 = (pow(y, r, p) * pow(r, s, p)) % p
    return v1 == v2

p = 467
g = 2
message = "Authorize Payment"

print("--- ElGamal Keys ---")
private_key, public_key = elgamal_keygen(p, g)
print(f"Public Key (y):  {public_key}")
print(f"Private Key (x): {private_key}")

print(f"\n--- Signing Message: '{message}' ---")
r, s = elgamal_sign(message, p, g, private_key)
print(f"Signature pair (r, s): ({r}, {s})")

print("\n--- Verification ---")
is_valid = elgamal_verify(message, r, s, p, g, public_key)
print(f"Signature Valid: {is_valid}")
```

Input:

Plaintext

```
p = 467
g = 2
message = "Authorize Payment"
```

Output:
(Note: Key pairs and signatures will differ on each run due to randomness)

Plaintext

```
--- ElGamal Keys ---
Public Key (y):  182
Private Key (x): 244

--- Signing Message: 'Authorize Payment' ---
Signature pair (r, s): (347, 86)

--- Verification ---
Signature Valid: True
```